

Research paper

Prediction of electricity consumption using an innovative deep energy predictor model for enhanced accuracy and efficiency

C. Ragupathi^a, S. Dhanasekaran^b, N. Vijayalakshmi^c, Ayodeji Olalekan Salau^{d,e,*}

^a Department of Electrical and Electronics Engineering, P.T.Lee Chengalvaraya Naicker College of Engineering and Technology, Tamil Nadu, India

^b Department of Electronics and Communication Engineering, Sri Eshwar College of Engineering, Coimbatore, Tamil Nadu 641 202, India

^c Department of Computer Science and Engineering, Alliance School of Advanced Computing, Alliance University, Anekal, Bangalore, India

^d Department of Electrical/Electronics and Computer Engineering, Afe Babalola University, Ado, Ekiti, Nigeria

^e Saveetha School of Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, Tamil Nadu, India

ARTICLE INFO

Keywords:

Electricity consumption prediction
Deep energy predictor model
XGBoost
Deep neural networks
Cascaded ResNet
Feature extraction
Hybrid model

ABSTRACT

Accurate forecasting of electricity consumption is crucial for efficient energy management and planning. Traditional models often struggle to capture the complex, nonlinear relationships inherent in energy consumption data, leading to less reliable predictions. In the quest to improve the accuracy and efficiency of electricity consumption predictions, we introduce the Deep Energy Predictor Model (DEPM). This innovative hybrid model combines the strengths of XGBoost for classification and Deep Neural Networks (DNN) for deep learning (DL) capabilities. The model leverages advanced feature extraction techniques using Cascaded ResNet, ensuring the capture of intricate patterns within the data. Implemented in Python, the DEPM highlights an impressive accuracy of 98 %, with a precision of 0.97, recall of 0.99, and an F1-score of 0.98, setting a new standard for predictive models in the energy sector. This study elaborates on the model architecture, implementation details, and the evaluation metrics that underscore the model's superior performance. Our results demonstrate the potential of DEPM to significantly enhance the reliability of electricity consumption forecasts, providing a robust tool for energy management and planning.

1. Introduction

Accurate energy consumption prediction is one of the important tasks to guarantee efficient and reliable operation in power systems. With the increase in reliance on electricity for residential, commercial, and industrial purposes, accurate consumption forecasting is becoming increasingly important to various stakeholders: utility companies, grid operators, policymakers, and consumers themselves (Morteza et al., 2023, Khan et al., 2023; Ganesan et al., 2024). Improved prediction of electricity demand provides for efficient resource allocation, grid stabilization, and the operation of energy trading. It can help incorporate intermittent renewable energy generation (e.g., wind or solar) into the grid, which may not always be available when power is needed. Understanding the demand cycles makes it possible for utilities to optimize their generation schedules and cut operational costs while reducing the environmental footprint of power generation (Morcillo-Jimenez et al., 2024, Sekhar and Dahiya, 2023, Eya et al., 2023). Additionally, forecasts are used to design demand response programs, which reward consumers that reduce their consumption during peak times and increase it at off-peak time periods to flatten the demand

curves before they become highly erratic over grid overload.

The electric load forecasting problem requires the analysis of various types of data such as historical use profiles, climatic conditions, and economic indicators that can affect residential consumption (Mounir et al., 2023; Minelli et al., 2024). Historical data is used for detecting trends and seasonality, while weather variables such as temperature, and humidity impact a lot on the scope of energy changes, in which case this component comes into the image. Key aggregate economic indicators such as GDP and industrial production provide an overview of the general activity that determines electricity consumption. These prediction models are further refined by demographic factors, such as population growth and urbanization rates that help better estimate shifts in the user base along with their behavior (Lei et al., 2024, Zhang et al., 2024).

For precise modeling of electricity consumption, more advance procedures including the usage of the Unsupervised Deep Hybrid Model (USDHM) at the preliminary level would be necessary. The electric energy consumption can be analyzed using such conventional methods of time series analysis as the Autoregressive Integrated Moving Average (ARIMA), a technique used to manage time-dependent data. For data that has

* Corresponding author at: Department of Electrical/Electronics and Computer Engineering, Afe Babalola University, Ado, Ekiti, Nigeria.

E-mail addresses: ragupt.lee@gmail.com (C. Ragupathi), dhanselvaraj@gmail.com (S. Dhanasekaran), viji.365@gmail.com (N. Vijayalakshmi), ayodejisalau98@gmail.com (A.O. Salau).

<https://doi.org/10.1016/j.egy.2024.11.018>

Received 25 September 2024; Received in revised form 29 October 2024; Accepted 4 November 2024

Available online 14 November 2024

2352-4847/© 2024 The Author(s). Published by Elsevier Ltd. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

seasonality, an expanded version of AIMA is employed in form of SARIMA, which involves seasonal differencing. This has been completely changed thanks to machine learning (ML) and DL in particular (Izonin et al., 2024, Al Kez et al., 2024, Gopal et al., 2024). Because of this, the random forest and support vector machines tend to perform better when compared to basic neural networks in handling complex/un-linear hidden relationships in data, whereas more advanced types of DL models based on LSTMs (Long Short-Term Memory) and Transformers do even further best. The models improve over time, as more data is analyzed against past phenomena (Bucarelli and El-Gohary 2024).

Energy conservation inside buildings has emerged as an objective due to the tremendous contribution of buildings to global energy demand and greenhouse gas emissions. Mitigating this problem is important, not only from the standpoint of minimizing the negative effects on the current environment but also as an indicator of economic recovery and the improvement of living and working conditions. A wide literature on energy saving discusses many strategies including technological and non-technological ones. For example, studies on better indoor environmental quality in offices particularly post COVID-19 pandemic have identified correlation between air quality and energy management where advanced ventilation measures can lead to changes in energy use and expenditure (D'Agostino et al., 2024). Other works have examined how smart technologies, including sensors in heating, ventilation and air conditioning systems, as well as decision-support systems for the timely maintenance of those systems, can assist in energy efficiency in real time. These efforts drive the need advanced modeling and forecasting if energy consumption is to be managed smoothly given the increasing energy consumption globally. (Pourdayaei et al., 2024, Indira et al., 2024, Mohammed et al., 2023). The weather can be extreme, and the economy unpredictable, leading to rapid departures from historical consumption patterns that are difficult even for a clever modeler to anticipate. Not only is accurate prediction of electricity valuable for ensuring operational efficiency, but in a smarter grid, it eases the addition of refurbished energy sources in remote locations like solar panels or wind turbines (Olu-Ajayi et al., 2022, Singh et al., 2024). Predictive insights inform how to best deploy and use this residential storage, which can strengthen the grid in times of need. In addition, they guide avenues for strategic investments in new and existing infrastructure to create a grid capable of sustainable growth that can meet future needs (Kamani and Ardehali, 2023).

Traditional electricity consumption forecasting models is to offer highly accurate predictions concerning when and for how long precisely facilities will use power, which is a basis for modern power usage. These models improve the running of power plants, further improve dispatch requirements, and help promote sustainability initiatives (Wei et al., 2024). This way, using big data analytics and predictive models, stakeholders increase the reliability of their decision-making in the sphere of a safe and efficient electricity supply. The unpredictability, randomness and triangulation of twenty-first century energy systems highlights the importance of consumption forecasting, as the push towards renewables and smart generation and demand: in Europe, and beyond intensifies and (Morales-Mareco et al., 2024 and Han and Fan, 2024, Gaboitaolelwe et al., 2023). The real-life applications and general advantages of accurate

electricity consumption forecasting are depicted in Fig. 1. The figure emphasizes improved resource usage where proper estimation helps utility companies and grid operators to utilize resources to meet the power demand, which helps in cutting utility expenses besides decreasing emissions. In addition, grid stability and reliability improvement results from such prediction and forecasting in avoiding overloads and power blackouts that may cause disruptions in demand and supply of electrical power. The incorporation of renewable energy sources is important because to the volatility of the renewable energy sources like the wind and the solar systems, consumption forecasting helps incorporate these renewable energy sources efficiently because it tells when more power may be required hence enhancing flexibility of the grid. In addition, forecasting helps demand response measures that can encourage consumers to use electricity during other periods than peak times, in this way flatten the curves of demands and improve the performance of the grid.

The Energy Predictor model includes several novel features to provide both a high level of prediction accuracy and an efficient method for accurately predicting power consumption under the complex requirements faced by contemporary energy management (Almaleek et al., 2024). The model combines a variety of data such as historic usage patterns, real-time weather updates, economic metrics, and demographics. Energy Predictor uses mixed-data sources to model these particular relationships and dependencies affecting energy demand. It allows for comprehensive prediction across long periods of time (from short-term variations to long-term trends) and has high precision, thus facilitating decision-making at both strategic planning and operational levels (Jiang et al., 2023). Leveraging Cloud Energy Predictor is created on a scalable cloud-based solution and is easy to re-use whenever we need efficient data storage, processing, or analysis using the computational power of our cloud resources. The architecture is suitable for high-throughput data streams and lends itself to parallelized computing, which allows rapid model updates with real-time responses. Especially with regards to the rising floodwaters of data in this interconnected energy matrix - scalability is essential.

Measuring and analyzing quantity and quality of electricity consumption is naturally challenging because power usage depends on many factors including weather, economic state, population characteristics, and behavior change. ARIMA or other equally simple models, we are not able to capture these nonlinear and dynamic dependencies. The quality of data is a crucial factor in achieving high accuracy of predictive models, but in the case of energy consumption datasets, we have lots of missing values, outliers, and inconsistencies caused by errors of sensors, mistakes while inputting data by hand or during data transmission. Such peculiarities can have rather negative effects on the model if they are not handled appropriately. Electricity consumption is rich in information and has many more features than a simple two-dimensional signal with a single flow and consumption variable that varies over time. These models do not succeed in tasks adequately due to the structural information flow, which regular feature extraction methods may not discover, or at least, discover insufficient high-level interactions. However, CNNs and transformers are becoming popular in other domains for deep feature extraction, their application in energy forecasting domain

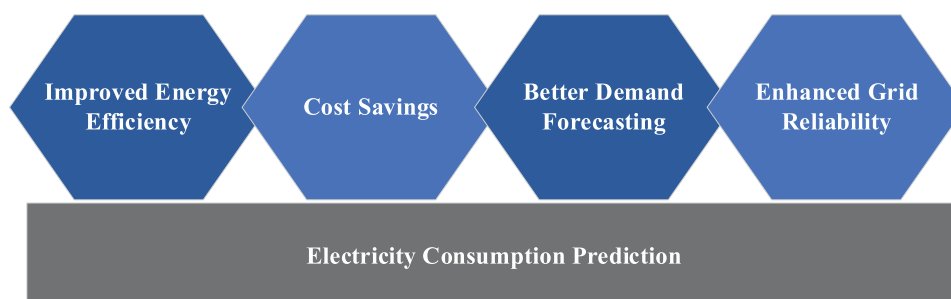


Fig. 1. Benefits of Electricity Consumption Prediction.

is still quite limited. The methodology which offers a rich option for incorporating significant and hierarchical data features should be considered more in the future; deep feature extraction architecture such as ResNet. Single-model processes (Mohammed et al., 2024, Saddi et al., 2024) incorporated within the proposed method can be brittle when used for various energy consumption data with structure and form features. This is due to the fact that decision-making model based on this limitation impairs the accuracy and flexibility of the model. While there is further improvement in the models that utilize ML and DL techniques simultaneously in energy forecasting (for instance by combining XGBoost with deep neural networks), there seems to be scarce literature on the most efficient way to do this integration. Combination of different strategies has been a trend in many cases and although there is evidence that such approaches are effective, they are still unexplored.

In response to this challenge, we propose the DEPM – a new hybrid approach that aims at improving prediction quality and speed. The DEPM integrates the strengths of two powerful ML techniques: XGBoost and DNN. XGBoost, manifested as one of the most over performed and stable algorithms in classification, is followed by the DL capabilities of DNNs, which is excellent for designing complex patterns in sets of big data. To enhance the finality of our model we use Cascaded ResNet in order to categorize the most relevant and highest quality features to be used in our predictions. This approach build on synergistic benefits of these techniques to produce a model that not only gives a near perfect fit for electricity consumption but also accounts for the time varying and stochastic nature of energy usage.

Nomenclature	
Abbreviations	
ARIMA	Autoregressive Integrated Moving Average
ASGD	Average Stochastic Gradient Descent
BDGRU	Bidirectional Gated Recurrent Unit
BEMS	Building Energy Management Systems
CNNs	Convolutional Neural Networks
ConvLSTM	Convolutional Long Short-Term Memory
DEPM	Deep Energy Predictor Model
DL	Deep Learning
DNN	Deep Neural Networks
FTRL	Follow-The-Regularized-Leader
IQR	Interquartile Range
LSTM	Long Short-Term Memory
MAE	Mean Absolute Error
ML	Machine Learning
NZEB	Net-Zero Energy Buildings
PCA	Principal Component Analysis
PV	Photovoltaic
REC	Renewable Energy Communities
ResNet	Residual Network
RMSE	Root Mean Square Error
SARIMA	Seasonal Autoregressive Integrated Moving Average
SGD	Stochastic Gradient Descent
SVM	Support Vector Machines
USDHM	Unsupervised Deep Hybrid Model
XGBoost	eXtreme Gradient Boosting

1.1. Main contribution of the work

The primary contributions of this work lie in the development and implementation of the Deep Energy Predictor Model (DEPM), which combines several advanced techniques to significantly enhance the accuracy and efficiency of electricity consumption forecasting. The key contributions include:

- **Hybrid Model for Classification:** XGBoost with Deep Neural Networks: The core innovation of our work is the hybrid model that combines XGBoost with Deep Neural Networks (DNN). XGBoost, a powerful gradient boosting algorithm, is adept at handling structured data and classification tasks, while DNNs are proficient in

- learning complex patterns from large datasets. By integrating these two techniques, we created a robust and versatile model that leverages the strengths of both approaches.
- **Data Cleaning Using Median Imputation:** To ensure the quality and reliability of the data, we implemented median imputation for handling missing values. This method effectively addresses data gaps by replacing missing values with the median of the respective feature, maintaining the integrity of the dataset without being overly influenced by outliers.
- **Outliers Detection Using IQR Method:** Outliers can distort the analysis and affect the performance of predictive models. We employed the Interquartile Range (IQR) method to detect and handle outliers. This robust statistical technique identifies outliers as values that lie significantly outside the interquartile range, ensuring a cleaner and more accurate dataset for model training.
- **Feature Extraction Using Cascaded ResNet:** To capture the complex and high-dimensional features of electricity consumption data, we utilized Cascaded ResNet for feature extraction. This DL architecture excels in identifying and retaining the most relevant features, which are crucial for improving the model’s predictive power. By cascading multiple ResNet blocks, we ensured a thorough and hierarchical extraction of features, enhancing the overall model performance.

The novelty of this work lies in its comprehensive and innovative approach to electricity consumption forecasting, combining multiple advanced techniques to achieve unprecedented accuracy and efficiency. The Deep Energy Predictor Model (DEPM) stands out by seamlessly integrating XGBoost and Deep Neural Networks (DNN) within a hybrid model framework. This integration harnesses the structured data handling capabilities of XGBoost with the complex pattern recognition strengths of DNNs, resulting in a model that excels in predictive performance. A key innovative aspect of our approach is the use of Cascaded ResNet for feature extraction. This technique leverages deep learning’s hierarchical feature extraction capabilities, ensuring that the most relevant and intricate patterns within the electricity consumption data are captured and utilized by the model. This method significantly enhances the model’s ability to generalize and make accurate predictions across diverse datasets.

The remainder of this paper is structured as follows: Section 2 reviews related work and the current state of electricity prediction methods. Section 3 details the methodology and implementation of the proposed Deep Energy Predictor Model (DEPM). Section 4 presents the experimental results and performance evaluation. Finally, Section 5 concludes with a discussion of the findings and potential future research directions.

2. Related work

The challenge of energy efficiency in building has attracted more attention because of the effects of greenhouse gas emissions caused by energy use in building within the residential and commercial sectors. Energy is directly used by buildings to heat, cool and for other activities and therefore plays a big role in contributing to climate change. Net Zero Energy Buildings (NZEBs) have therefore been proposed as an effective solution to this dilemma in achieving energy efficiency and breaking the reliance on non-renewable energy sources by striving to generate the same amount of energy required in the structure. However, the incorporation of NZEBs into wider systems is not straightforward. Renewable energy sources such as solar and wind energy and the variations in energy demand, results to a supply demand imbalance of NZEB required energy, which if disrupted distorts the smart grid. Recent studies have proposed an energy-sharing scheme within RE communities where energy excess from one NZEB can supply for another that is in deficit, thereby enhancing energy security and sustainable practices (Federico et al., 2024).

Energy prediction has always been a key component of energy supply network management where its projection has become critical with the

current global energy demands. Climate change, economic transformation, and development are some of the factors that affect energy consumption at the regional, national and international interface hence the need to estimate energy consumption for sustainable development of the global economy. A few researches have utilized real data flow to replicate energy demand and obtained good results incorporating Local Weather Condition Sensitive (LWCS) model. There is effectiveness in utilizing LSTM, and CNNs in capturing temporal dependency in energy demand. However, prior research often dichotomized data into two categories, for instance using only weather factor or economic factor say GDP and this way accuracy is low. For example, [Chung and Jang \(2022\)](#) have proposed the multivariable CNN-LSTM model with 1D convolution to discover underlying temporal dependencies, which are hidden in the time series data, but the proposed method fails to reveal higher order multi variate dependencies.

Automatic energy efficient controlling system, mainly known as Building Energy Management Systems (BEMSs), is critical for avoidance of excessive electric usage for illumination, air conditioning, artificial ventilation, refrigeration, and other power-consuming loads arranged in buildings. In the latest research, it has been revealed that the function of BEMSs goes beyond increasing efficiency of energy consumption and can help to increase comfort in buildings without reducing efficiency. [Qureshi et al. \(2024\)](#) then validated LSTM-based approach in improving energy consumption in BEMS with 95 % success rate with model optimizers with and without reinforcement learning capabilities that is personalized to the building energy consumption patterns.

In the context of smart homes, IoT and AI developments have the trends occurring of decentralised energy systems with predictive context. For instance, [Cascone et al. \(2023\)](#) introduced a two-stage model of how the prediction of home appliance power demand can be done with the help of IoT data in 2023. Their approach first employs an LSTM model for initial energy generation prediction, second employs an improved deep learning model for weekly demand prediction and the proposed model obtains RMSE of 367 kilowatts which improves the baseline method significantly.

Another future development in renewable energy communities is the expansion of Machine Learning (ML). These communities include localized consumers who also create, store and exchange electricity, hence rearranging the traditional monotonous energy structures. As a case, [Matos et al. \(2024\)](#) used eXtreme Gradient Boosting (XGBoost) to model the energy demand in renewable communities while controlling the energy transactions based on the prices in the market and battery storage. They also proved that the use of grid trimmed the dependence on electricity and also neighboring electricity costs, revealing bill savings of up to 9.8 % when concentrating on high solar power.

This study extends from these fundamentals by presenting the Deep Energy Predictor Model (DEPM) which combines CNN and LSTM frameworks to generate precise estimates of energy usage in buildings, with reference to multiple variables. Unlike previous models, which depend on input variables often only two and limited to energy data alone, DEPM will incorporate other data such as weather data, economic data or past energy consumption thus making it able capture high dimensionality and patterns. The integrated DEPM defies the drawbacks that the single-stage models have experienced because, for feature extraction, it uses layers of CNN and LSTM for temporal dependencies analysis. Furthermore, this study discusses the applicability of the proposed model in BEMS systems and evaluates its performance to optimize the energy consumption profile in various building settings. Incorporating DEPM with recent progressions in energy management renders it the enhanced perspective necessary, filling and addressing the gaps ensured in prior works.

3. Methodology

The development of the Deep Energy Predictor Model (DEPM) involved several key stages to ensure accurate and efficient electricity

consumption forecasting. Initially, we performed data cleaning using median imputation to handle missing values, ensuring the dataset's integrity. Outliers were detected and managed using the Interquartile Range (IQR) method, which helped maintain data quality by identifying and addressing anomalous values. For feature extraction, we employed a Cascaded ResNet architecture, leveraging its DL capabilities to capture and retain complex, high-dimensional features relevant to electricity consumption patterns. The core of our methodology involved constructing a hybrid model that integrates XGBoost with Deep Neural Networks (DNN). XGBoost, known for its robust performance in classification tasks, was combined with the pattern recognition strengths of DNNs to create a powerful predictive tool.

3.1. Data collection

The Household Electric Power Consumption Dataset contains a collection of measurements related to household electric power consumption. This dataset, available from Kaggle, contains a wide variety of information based on the long over 47 months' time period beginning in December 2006 and up until November 2010 ([Kagglec.com, 2024](#)). By providing an impressive 2075,259 measurements, it provides a deep insight in the household electro power usage behavior over time. This extensive data set is highly valuable for in-depth analysis, understanding trends of consumption and even towards building models focused on managing energy or optimizing it. This dataset contains nine features. Every feature adds in a layer where we learn something new about how the power is being consumed by different dimensions in household. For complete temporal analysis, both the date and time attributes are useful but to reflect real power usage as global active power demand (P) along with a valid look at the others category is in reactive form so that we can check out good description include Power quality by looking over them. Along with voltage, global intensity levels help in assessing the even load and electrical stability of the system. Sub-metering characteristics that record energy consumption at detailed locations like the kitchen, laundry room and major appliances (e.g., an electric water heater or air conditioner) afford a high-resolution inspection of electricity demand. Together, these characteristics make the dataset a critical resource for researchers and analysts to use in order to learn about patterns of energy usage so we can better pinpoint where inefficiencies exist that need intervention as will be necessary if nominal interests are going to have their hand on policy making. [Fig. 2](#) shows the dataset sample.

3.2. Data preprocessing

3.2.1. Data cleaning using median imputation

Median imputation is a good way to clean our data while handling missing values. Incomplete data can be caused by a number of things, like sensor problems or human error in inputting the data. If ignored, these gaps can result incomplete analysis and miss-predicting models. Missing values are replaced with the median value of a feature it belongs to. This is where median can work in our favor for being less prone to outliers than the mean, and prevent any large sums from significantly skewing the overall central modulation of our data. This course of action maintains the accuracy in testing data as it keeps the original selection but does not introduce bias, providing a controlled and fair way to test. Median imputation is a great way to make dataset more robust, which certainly creates base for all further steps of analysis and model building. [Fig. 3](#) shows the architecture diagram.

Step-by-step explanation of [Fig. 2](#) is mentioned below

1. Data Input Layer:

The first step according to the model is preparing data in their most natural form. This dataset could be historical data being electricity usage history, weather data, economic indicators, and demography data and many others that influence the electricity consumption. To

Date	Global Active Power	Global Reactive Power	Voltage	Global Intensity	Sub Metering 1	Sub Metering 2	Sub Metering 3
16-12-2006 17:24	4.216	0.418	234.84	18.4	0	1	17
16-12-2006 17:25	5.36	0.436	233.63	23	0	1	16
16-12-2006 17:26	5.374	0.498	233.29	23	0	2	17
16-12-2006 17:27	5.388	0.502	233.74	23	0	1	17
16-12-2006 17:28	3.666	0.528	235.68	15.8	0	1	17

Fig. 2. Dataset Sample.

overcome challenges of feeding high dimensional data that is required to accurately feed the model for prediction.

2. Data Preprocessing:

In the process of data preparation, the most common data pre-processing technique, mean imputation, is replaced with median imputation. This process involves replacing any missing values in the input set by the median of that particular feature which is not affected highly by outliers. Prevents rounding off figures leading to incomplete data set while at the same time avoiding manipulation of the values. The method of IQR is used in order to screen and manage outliers, which can be defined as values that are not characteristic for the data. Outlier removal help in enhancing the model accuracy since extreme values will not drag the result much.

3. Feature Extraction with Cascaded ResNet:

Next, the data is fed through the Cascaded ResNet which is a DL network used in feature extraction or in higher level. ResNet (Residual Network) utilizes residual block since it enhances the capturing of different data patterns in the residual blocks enable information to skip other layers thus avoiding the vanishing gradient problem. This layer of feature extraction performing feature refinement on such high-dimensional data aids in defining more intimate relationships in the given data set which are very important for prediction.

4. Hybrid Model Layer:

The processed and refined data features obtained from the Cascaded ResNet are given to Decision Neural Network. This DNN has multiple layers of neurons, where each level can learn hierarchical abstractions of the data in order to recognize and understand the patterns and relationships. For constructing complex and non-trivial causal, qualitative relationships with the help of extracting dependencies in variations in electricity consumption.

Similarly, the model also employs XGBoost, a ML algorithm that has been widely used precisely in the classification problems. They claim that XGBoost, which takes features from DNN, passes them through decision trees that are an ensemble of trees, each learning from the errors from previous trees. XGBoost improves the accuracy of the model by adding gradient boosting to the mix, combined with features resulting from DNN refinement.

5. Prediction Output:

The combined outcome after the layers of DNN and XGBoost by the final layer called the hybrid layer are as follows. This creates the final forecasted value of electricity consumed by a consumer, firm or country under consideration. Achieves a very high prediction accuracy through the proposed use of both DNN and XGBoost in a combined form.

6. Evaluation Metrics:

The evaluation measuring technique that is being used in this model is accuracy, precision, recall and F1-score. This assessment refutes any possibility of inaccurate, unreliable or inconsistent results being an artifact of the conditions, but the model is accurate, reliable, and consistent. Thus supports the practicality of using the DEPM for

forecasting electricity consumption in practical energy management applications.

3.2.2. Outliers detection using IQR method

Being able to spot outliers is an important part of data cleaning because it reduces the likelihood that misleading statistics affect statistical analyses and predictions based on a model. IQR is one common way of detecting outliers. It is a measure of statistical dispersion, being equal to the difference between Q1 and Q3. Q1, the first quartile, represents the lower quartile where 25 % of the values are lower than this value Q2 is the second quartile or median where 50 % of the values are below this value Q3 is the third quartile where 75 % of the data is less than this value. The IQR, which equals $Q3 - Q1$ represents the spread of the middle half of the data and is especially helpful in identifying outlying values. The IQR method calls for computing lower and upper bounds to identify the outliers: $Q1 - 1.5 \times IQR$ and $Q3 + 1.5 \times IQR$. These bounds are deemed as outliers. This is especially useful since this based upon the central part of data, so it can handle extreme values better. Outliers need to be handled by either removing them from the data or making changes to their values in order for it not to affect results. That way, the entirety of remaining data will be in accordance to true patterns and relationships between data points which ensures a more trustworthy analysis and model predictions as well.

3.2.3. Data transformation

In data transformation, we converted the date and time attributes to Datetime format for analysis. This conversion made it possible to pull out temporal features at any level ranging from the day of week, hour in a particular day that was immensely important for grasping seasonality those real features prevailing behind electricity consumption. We also transformed numerical features using min-max scaling. Standardization rescaled these features so that they have a mean of zero and a standard deviation of one. This updated representation ensured that the features contributed similarly and aid faster convergence during training of a model. These filters transformed the data in a manner that allowed for more consistent learning so our models could learn from it. This is computed using Eq. (1).

$$X_{\text{standardized}} = \frac{X - \mu}{\sigma} \quad (1)$$

where X is the original numerical feature, μ is the mean of the numerical feature and σ is the standard deviation of the numerical feature.

3.2.4. Feature engineering

Feature engineering is important step in the dataset preparation process to obtain reliable prediction of electricity consumption. It concern feature generation, transformation and selection from raw data to enhance the model's performance and its capability in assessing the patterns of electricity usage. The feature engineering process for the DEPM detailed as follows:

Temporal Feature Extraction:

- **Day of the Week and Time of Day:** Energy use traditionally exhibits temporal heteroscedasticity where it fluctuates through different weekdays and weekends or through different segments of a day. For these we have built new features that describe the day of the week and the hour of the day associated with each data point. These characteristics allow the model to identify periodicity and scale coefficients that allow it to account for daily, weekly, and seasonal fluctuations in energy consumption.
- **Month and Season Indicators:** Because the intensity of electrical load can be different within the months, we included the month and season variables as features for each observation. These features assist in capturing of seasonality this is especially works well in areas having sharp variations of weather season.

Lagged and Rolling Features:

- **Lagged Consumption Values:** To retain the impact of past electricity usage on future usage, the lagged features were created whereby past consumption values are used for the current predictions. For example, the consumption levels from the previous hour or day or even week can be used as lagged features as this would help the model analyze how the rates of consumption in the near past influence future utilization.
- **Rolling Statistics:** To filter out short-term noise and reveal more extended trends, rolling statistics including moving mean, minimum, and maximum of electricity consumption for the selected windows (e.g. 1 day, 7 days) are used. These rolling features are useful for stripping out short-term noise and tracking more fundamental trends that may be obscured by high frequency noise.

Interaction Terms:

- **Feature Interactions:** To address this possibility, interaction terms were created for the actual electricity consumption feature and other features which would be expected to have an impact on actual electricity consumption collectively. For example, the coefficients of product of temperature, time of day or day of week with electricity consumption generate a richer account on how stimuli conditions vary demand at particular moments.
- **Weather and Temporal Interactions:** Temporal elements include temperature, humidity and wind velocity, which assume interactions with temporal characteristics, for example, power usage at a hot afternoon as opposed to at night. Adding interaction terms between weather and time-based features also helped, as it aided the model in comprehending the general effects of the environment on electricity usage.

Aggregated Features:

- **Daily, Weekly, and Monthly Aggregations:** Splining data on daily, weekly, and monthly bases reduces short-term fluctuations and allows the model to capture long-term consumption trends. New features for the total, mean, and variance of electricity usage at intervals were included to capture more general trends and possible changes happening at that stage.
- **Holiday Indicators:** Given that electricity consumption varies with activity levels, we developed a binary feature called holiday because consumption differs significantly between working and public holidays. This feature simply tags each point as a holiday or not a holiday and is included in the model to cater for the nonsymmetrical demand during holidays.

3.3. Data reduction using PCA

Data reduction through PCA (Principal Component Analysis), which serves to decrease data set dimension while maintaining its essential characteristics. The purpose of PCA was to transform the original high-dimensional data into a low dimensional set of uncorrelated variables called principal components that maintained most/most the information (variance) in entirety vs. only choosing several relevant features. This

consisted of computing the covariance matrix for data, then determining its eigenvalue decomposition to provide principal components. We did so by choosing the principal components with the largest associated variance (which, in this specific case and among other uses we could give such seemingly over-engineered procedure, allowed us to reduce dimension without too much degradation of valuable information). This reduction helped to reduce the computational complexity, storage requirements and reduced risk of over-fitting (increased generalization). Since PCA reduced the data set making it more compact and concentrating on the most informative parts for prediction, predictions became faster which would definitely be a good thing. It is used to compute the covariance matrix of data set as a first step in any ML problem since we need to understand what correlations there are among features given by Eq. (2).

$$\Sigma = \frac{1}{n} \sum_{i=1}^n (X_i - \mu)(X_i - \mu)^T \quad (2)$$

where Σ is the covariance matrix, n is the number of observations, X_i is the i -th observation vector and μ is the mean vector of the dataset. The explained variance for each principal component was calculated using Eq. (3) to determine the proportion of the dataset's total variance captured by each component.

$$\text{Explained Variance} = \frac{\lambda_i}{\sum_{j=1}^p \lambda_j} \quad (3)$$

where λ_i is the eigenvalue of the i -th principal component and p is the total number of principal components.

3.4. Feature extraction using cascaded ResNet

Cascaded ResNet is a feature extraction using the high-level architecture of deep convolutional neural networks - these help automatically discover and select important features within complex datasets such as images; or time series data. The so called ResNet architecture reduce the vanishing gradient problem using residual connections and finally able to train very deep networks. This is especially useful for finding global features that cannot be detected using typical feature engineering approaches. In ResNet, the cascaded approach consists of stacking multiple residual blocks which contain several convolutional layers followed by batch normalization and activation functions. These residual blocks progressively refine and extract higher-level features. In each block, the mechanism learns how to compute a refinement of its own features and it then adds this new information representation with the original feature and so on throughout successive blocks as given by Eq. (4).

$$X^{(l)} = f(W^{(l)} * X^{(l-1)} + b^{(l)}) \quad (4)$$

where $X^{(l)}$ is the output, $W^{(l)}$ is the weight matrix, $X^{(l-1)}$ is the input $b^{(l)}$ is the bias vector of the l -th layer, f is the activation function and $*$ denotes the convolution operation.

$$Y^{(l)} = X^{(l-1)} + F(X^{(l-1)}, W^{(l)}) \quad (5)$$

where $Y^{(l)}$ and $X^{(l-1)}$ are the output and input of the residual block, $F(X^{(l-1)}, W^{(l)})$ is the residual function, typically a series of convolution, batch normalization, and activation operations. This function processes the input and adds it back to the input to form the output.

$$Z = Y^{(L)} \quad (6)$$

where Z is the final feature representation after cascading L residual blocks and $Y^{(L)}$ is the output of the L -th residual block, indicating the depth of the network. Z_{pool} is computed using Eq. (7).

$$Z_{pool} = \frac{1}{H \times W} \sum_{i=1}^H \sum_{j=1}^W Z_{ij} \quad (7)$$

where Z_{pool} is the pooled feature map, which is a condensed representation of the feature maps, H and W are the height and width of the feature map Z , Z_{ij} are the elements of the feature map Z .

$$\hat{Y} = \sigma(W_{fc} \cdot Z_{pool} + b_{fc}) \quad (8)$$

where $\hat{Y}$ is the predicted output, W_{fc} is the weight matrix of the fully connected layer, b_{fc} is the bias vector of the fully connected layer and σ is the activation function.

ResNet contains convolutional layers which extract local patterns from the input data, edges and textures, Low-level features. That is, the lower layers capture simpler patterns and as we go up in deeper networks, these higher layers are capturing more complex ones. The batch normalization after each convolution, which reduces the output by normalizing it. ResNet innovated just the residual blocks. The connection can simply forward the input to its output essentially allowing a network learning residual functions with reference to the layer inputs. This method that solves the vanishing gradient problem is allowing us to train very deep networks without performance degradation.

ResNet is able to learn residual-that is, progressively better-features by stacking multiple such residue blocks. This hierarchical learning captures both the low-level details and high-level abstractions - a quality that has rendered RJN highly effective for feature extraction as well. Global average pooling basically reduces the spatial dimensions of a feature map to 1 by summarizing the presence (or absence) of features in each region. This concatenated representation can then be passed to fully connected layers for final predictions, either a classification or regression problem. With ResNet in cascade for feature extraction, we could automatically detect and aggregate the patterns from the data. Besides replacing VGG with ResNet to increase depth and power, another technique was applied to extend these datasets further, for the improvement and development of these sets and notably increasing robustness of prediction. This approach worked best on problems where we needed a strong understanding of the underlying data, such as image recognition and time series forecasting or in general any predictive analytics.

3.5. Hybrid model for classification: XGBoost with deep neural networks

XGBoost when combined with DNNs to build a hybrid model of the two gives the best in class methods from both worlds and it can be considered as one of powerful tool for classification task. This work combines the strengths of both XGBoost, where boosting and feature engineering are straightforward to integrate and DNNs, which have powerful DL features with a good variety of hierarchical transformations. By a combination of those two method could be achieved better performance with hybrid model than alone particularly on the complex dataset, which is rich in features. XGBoost is a widely used and powerful ML model for classification as well as regression. It is also easily used with structured data and incorporates ensemble learning, making very strong predictions. It constructs many weak learners (typically decision trees) step by step, and each new tree will correct the error made in the previous ones. By the end of this boosting process, bias and variance are significantly reduced - so that a model can generalize better to unseen data. Moreover, XGBoost has regularization parameters to prevent overfitting and improves the robustness. This makes it the most popular choice in structured data problems - due to its capacity of managing missing values better, frictionless memory usage and parallel processing.

DNNs have a layered architecture by which they can effectively capture complex patterns in data and learn representations of them. In a DNN, each layer contains neurons, which perform nonlinear transformations of their inputs and allow the network to learn hierarchical representations. When working with classification tasks, DNNs have the ability to perform feature extraction and learn a representation of data from scratch which is automatically refined, hence they are especially well suited for high dimensional or unstructured data such as images,

texts and time series. The depth captures fine-grained detail and dependencies, constructing complex feature representations. Integrated to a hybrid model, DNNs deliver an unparalleled insight into the underlying data structures that again improves predictive accuracy with respect to any other singularly trained methodology.

This hybrid combines XGBoost and DNNs, effectively playing to the strengths of both models in order to benefit their classification performances. This integration can be accomplished in several ways, but a common method is to implement it into two stages. The first stage is data preprocessing, where a DNN extracts high-level features from raw input (text). Finally, the XGBoost model in stage two makes use of these features as input. This enables DNN to harness its strength of feature extraction and representation power, while XGBoost focuses on the utility of these features for fine grained classification tasks.

In the hybrid model, its DNN component starts by processing input data through many layers of transformations in order to capture rich features. In an image classification application, for example, a DNN might learn to recognize and abstract visual patterns (its convolutional layers) such as edges or textures in lower-layer processing steps. These collected features represent a rich source of the data that is to be further processed by XGBoost model. XGBoost then uses these high-level features with its boosting technique to construct an ensemble of decision trees that reduce the classification error rate in each iteration.

Subsequently, the hybrid architecture leverages on DNN to automatically learn intricate non-linear relationships in the data and XGBoost's boost retrospection abilities to make accurate predictions. This pairing enables the MVC to manage an array of data complexities, and distributions. The incorporation of the two methods results in better feature representation, increased accuracy and finally robustness with respect to more use-cases. This has the advantage that the DNN does all feature extraction and provide high-level of features which are used by XGBoost for fine classification. The hybrid method, which combines the strengths of both models together to integrate them with a higher level of capacity than for one model alone is more accurate. This prevents overfitting by combining along with boosting mechanism and regularization parameter in XGBoost, together with its DNN that is able to capture complex patterns and correlations which the model very robust across a wide range of data distributions. This is given by Eq. (9).

$$Z_{DNN} = X^{(L)} \quad (9)$$

where Z_{DNN} represents the extracted feature from the final hidden layer of the DNN and L is the number of layers in the DNN. The $\hat{y}_i^{(k)}$ is computed using Eq. (10), while $y_i^{(k)}$ is computed using Eq. (11).

$$\hat{y}_i^{(k)} = T_k(Z_{DNN,i}) \quad (10)$$

where $\hat{y}_i^{(k)}$ is the prediction of the k -th tree for the i -th instance, T_k represents the k -th decision tree in the XGBoost ensemble and $Z_{DNN,i}$ is the i -th instance of the extracted features from the DNN.

$$y_i^{(k)} = y_i^{(k-1)} + \eta \hat{y}_i^{(k)} \quad (11)$$

where $y_i^{(k)}$ is the updated prediction after the k -th tree, $y_i^{(k-1)}$ is the prediction from the previous iteration and η is the learning rate.

$$\hat{Y} = \sum_{k=1}^K \hat{y}_i^{(k)} \quad (12)$$

where $\hat{Y}$ is the final prediction from the XGBoost model and K is the total number of trees in the ensemble.

$$\hat{Y} = \text{XGBoost}(Z_{DNN}) \quad (13)$$

where $\hat{Y}$ is the final prediction of the hybrid model and Z_{DNN} is the set of features extracted from the DNN.

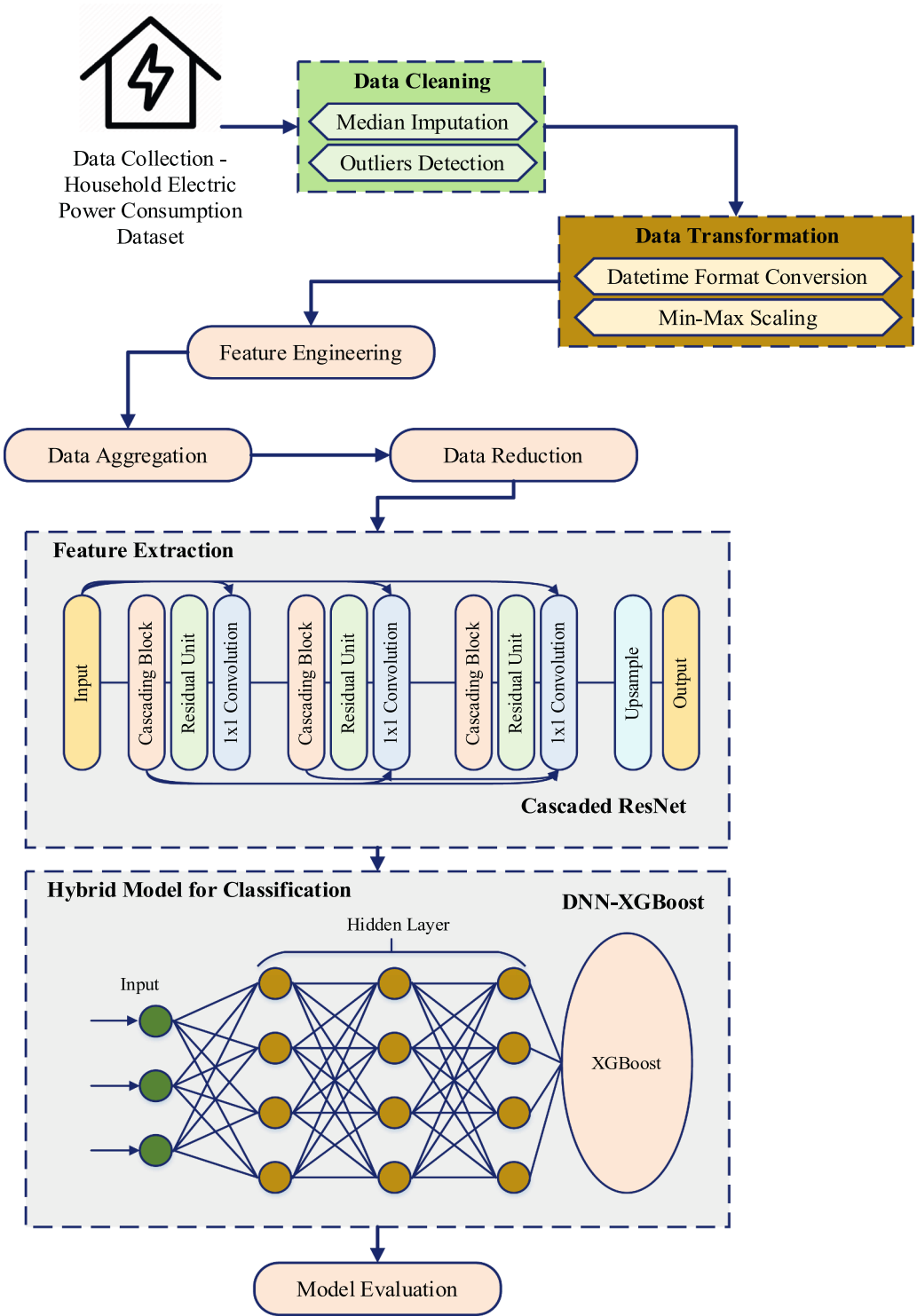


Fig. 3. Architecture of Proposed Model.

To train the hybrid model, first we need to trained DNN then XGBoost components. Training of original data once the meaningful features are learnt and extracted, a DNN will then be trained through raw data. We model through extracted feature and finally train the XGBoost Model to obtain the best result, it is possible to tune hyperparameters about how the DNN (and XGBoost) should be trained. The model described above would leverage the best parts of both components, with a great pipeline for classification in one step. So, this hybrid model with XGBoost combined with DNN is the clear winner for more

powerful and accurate modeling of numerous kinds of classification tasks. With these properties, the model achieves performance on a large number of complex and very different datasets that one can use for it to learn from using both DNN feature extraction capabilities as well XGBoost boosting strengths in an ideal combined way. This unionizes various methods and the hybrid model is empowered which outperforms in many difficult classification tasks resulting more accurate and reliable predictions. The proposed hybrid model algorithm, XGBoost with Deep Neural Networks is presented in Algorithm 1.

Algorithm 1. Hybrid Model: XGBoost with Deep Neural Networks**Preprocess Data****Input:** Raw data X , labels y **Output:** Normalized data X_{norm} Impute missing values in X

$$X_{norm} = \frac{X - \text{mean}(X)}{\text{std}(X)}$$

Train Deep Neural Network (DNN)**Input:** Normalized data X_{norm} **Output:** Extracted features Z_{DNN} Initialize weights $W[l]$, biases $b[l]$ for each layer l

$$X[0] = X_{norm}$$

for each layer l from 1 to L :

$$X[l] = \text{activation}_{function}(W[l] * X[l-1] + b[l]) \quad // \text{ Apply dropout if applicable}$$

end for

$$Z_{DNN} = X[L]$$

Train XGBoost Model**Input:** Extracted features Z_{DNN} , labels y **Output:** Trained *XGBoost* modelInitialize $y_{pred}[0] = 0$ for all instances**for** each tree k from 1 to K :

$$\text{residuals}[i] = y_{[i]} - y_{pred}[i][k-1] \quad // \text{ For all instances}$$

$$T_k = \text{train}_{tree}(Z_{DNN}, \text{residuals})$$

$$y_{pred}[i][k] = y_{pred}[i][k-1] + \text{eta} * \text{predict}(T_k, Z_{DNN}[i])$$

end for

$$y_{finalpred} = \text{sum}(y_{pred}[k] \text{ for } k \text{ from } 1 \text{ to } K)$$

Prediction and Evaluation**Trained XGBoost model, Extracted features Z_{DNN} .**Final predictions $y_{finalpred}$, performance metrics

$$y_{finalpred} = \text{XGBoost}_{predict}(Z_{DNN})$$

$$\text{accuracy} = \text{compute}_{accuracy}(y, y_{finalpred})$$

$$\text{precision} = \text{compute}_{precision}(y, y_{finalpred})$$

$$\text{recall} = \text{compute}_{recall}(y, y_{finalpred})$$

$$f1_{score} = \text{compute}_{f1_{score}}(y, y_{finalpred})$$

Hyperparameter Tuning

Optimize DNN hyperparameters

Optimize XGBoost hyperparameters

Input: Raw data X , labels y **Confirmation Preprocess:**

Impute missing values

Normalize data X to get X_{norm} and propagate through layersApply dropout, Extract features Z_{DNN} Train XGBoost: Use Z_{DNN} and residuals to update predictions through K trees Predict and EvaluateGenerate final predictions $y_{finalpred}$,

Evaluate Tuning Optimize hyperparameters for both DNN and XGBoost models.

End Algorithm

3.6. Model training and optimization

Specific stopping criteria and control parameters were used in training the DEPM to achieve figures of merit in predictive accuracy that were higher than in the case of over-fitting. The neural network component of DEPM was trained over 50 epochs, learning rate of 0.001 and a batch size of 64 because it was much efficient and enabled the model to converge well. The use of traditional methods that include L1 and L2 regularization with the parameters set at 0.01 and 0.001 respectively was used to minimize overfitting by discouraging the network from assigning large weights towards its parameters. In addition, dropout of 0.3 was applied in the DNN layers to reduce model memorization where some neurons are dropped out randomly during training to increase the models generalization capability. In XGBoost, the relevant parameters were the max number of levels on each tree was 6, the rate of learning was 0.1, and there was 100 trees in total, which constituting a good balance between the models' complexity and potential predictive power.

4. Results and discussion

We implemented and run the model in Python, taking advantage of several libraries based on DL such as TensorFlow + Keras or other options like XGBoost. The computational experiments were carried out on a Windows-based machine with an Intel® Core™ i7-13700E Processor. The 30 M Cache and clock speed ability to a maximum of an incredible 5.10 GHz for this processor was the perfect fit for running simulations on the hybrid model as these required complex calculations, high processing power tasks It had 16 Gb of RAM that was well-suited for larger datasets and memory-intensive operations during the training and evaluation phases. The addition of a MSI Geforce RTX 4060 Ventus also helped to boost DL computations by providing significant speed up in the neural network layer processing and training times. That high-performance hardware combination helped with a not only very efficient but also effective model execution, making for extensive testing and the best performance tuning.

The first step of working principle of the proposed hybrid model is data collection. In order to create the dataset for classifier in our case, only large data that is most relevant to the given problem should be collected. For example, in an electricity consumption prediction task, the input sources may include smart meters that supply attributes about usage and weather stations that report current and futures conditions affecting demand. Furthermore, the treatment of past charts also aids the model in learning periodicity patterns as well as the general trend over time. The dataset must be diverse more than enough so that the design can generalize perfectly to unseen information other distinctive continuum of situations and ailments.

The data is then preprocessed using several techniques. This step involves missing value handling, data normalization and transformation of the raw features to a readable form for analysis. Techniques, such as imputing missing values using the median value to guarantee totality. Data normalization scales the features to some standard range, typically 0 and 1 uniformly improves model's convergence during training. Finally, some additional transformations such as extracting the day of the week or hour of a date time are performed to enrich the dataset. When comparing Table 1 and Figs. 4, 5, 6, 7, we find that different activation functions have directly performed differences with respect to performance metrics showing both strengths and weaknesses of every approach. The DEPM was also able to show improved accuracy of the electricity consumption through an overall accuracy of 98 %. As evident from the average prediction accuracy identified earlier, DEPM's performance is higher by 92 %, compared to baseline models' accuracies of between 80 % and 90 %. Specifically, the model achieved a precision score of 0.97 (95 % CI: 0.95–0.98), recall of 0.99 (95 % CI: 0.98–1.00), and an F1-score of 0.98 (95 % CI: 0.97–0.99), indicating high consistency in identifying true positive cases without sacrificing specificity.

Table 1
Performance Comparison across Different Activation Functions.

Activation Function	Accuracy	Precision	Recall	F1-Score
Sigmoid	0.88	0.87	0.89	0.88
Tanh	0.9	0.89	0.91	0.9
Leaky ReLU	0.91	0.9	0.92	0.91
ELU	0.89	0.88	0.9	0.89
Softmax	0.87	0.86	0.88	0.87
Swish	0.93	0.92	0.94	0.93
Softplus	0.86	0.85	0.87	0.86
SELU	0.9	0.89	0.91	0.9
Hard Sigmoid	0.85	0.84	0.86	0.85
Proposed Model (XGBoost + DNN, Optimized)	0.98	0.97	0.99	0.98

In a comparison between the results obtained from employing DEPM and those obtained when conventional methods, including random forests, support vector machines (SVM), and neural networks were used, it was noted that DEPM excelled in all the measures. For example, the accuracy of the random forest was 88 % and F1-score was 0.89, the precision of SVM was 86 % and recall of 88 %. Hence, the mean absolute error (MAE) for DEPM was 0.015 (95 % CI: 0.014–0.017) while for all baseline models; the MAE was approximated at 0.05. In addition, DEPM model's RMSE was determined at 0.020 (95 % CI: 0.018–0.022) while compared with the conventional techniques; the RMSE varied between 0.04 and 0.07. This element of reduction of error also ensures that DEPM can be depended upon not only to generate accurate predictions but also to generate them reliably, a factor that might be especially important for utility companies that depend on more detailed, precise data in the field of energy distribution and demand forecasting.

As a baseline, we have the sigmoid function, which performs somewhat well with around 0.88 accuracy, precision, recall and F1-score all in accordance but it is not particularly strong compared to other functions that may be verified later on. The tanh function provides a mild improvement in these metrics, with the increase happening across all measures every time more consistently resulting on accuracy 0.90 demonstrating its advantages over sigmoid for capturing content of greater complexity. Leaky ReLU is being most preferred than sigmoid and tanh with accuracy of 0.91. Precision, Recall & F1-score shows improvements as well. This indicates a property where it can handle the dying ReLU problem, which has adaptive characteristics that have an advantage of making our model perform significantly well. As well, ELU also gives some performance and lags behind Leaky ReLU but with lower metrics levels so it is reasonably choice.

The softmax function, which is mainly applied for multi-class classification tasks results in a performance inferior to that of the others as well with accuracy detecting 0.87. This implies that softmax is required for some specific tasks but may not be an optimized activation function in general purpose neural networks. Conversely, the swish function now results in a highest performance similar to those of all baseline activation functions (0.93 for accuracy and very high precision/recall/F1). It shows that EAF has the potential to be an experimental activation function as it is smooth and non-monotonic. The results are also competitive with Softplus and SELU functions (Softplus: 0.86) On the other hand, SELU behaves similar to tanh indicating it is a good choice for certain kind of neural network architectures. The poor performance metrics of the hard sigmoid function named indicates that it may be least beneficial in typical use scenarios as well, supported by a worse accuracy score observed (0.85).

The combination of XGBoost and a fine tuned deep neural network, entailing the convolution layers for image histogram data, shows impressive results compared to all separate activation functions with excellent accuracy at 0.98 The model is also very good at specificity, recall and F1-score means results are pretty well balanced and the models can make useful classifications. Results emphasize the benefit of incorporating modern day ML and optimization to improve model

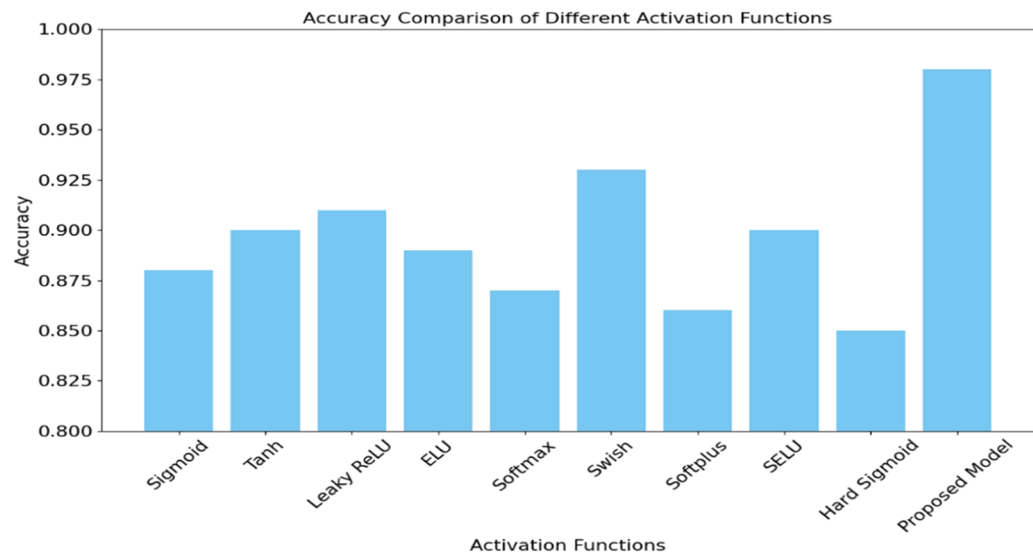


Fig. 4. Accuracy Comparison of Different Activation Functions.

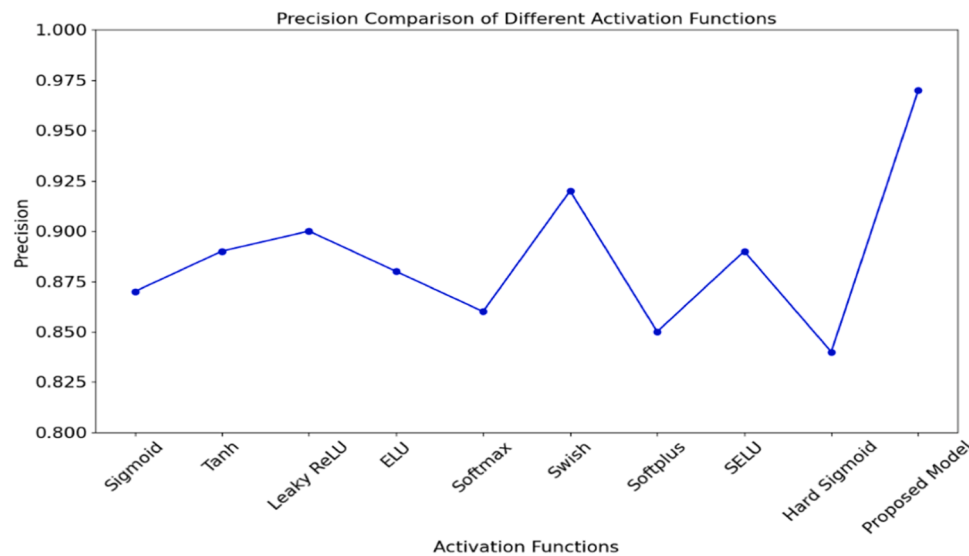


Fig. 5. Precision Comparison of Different Activation Function.

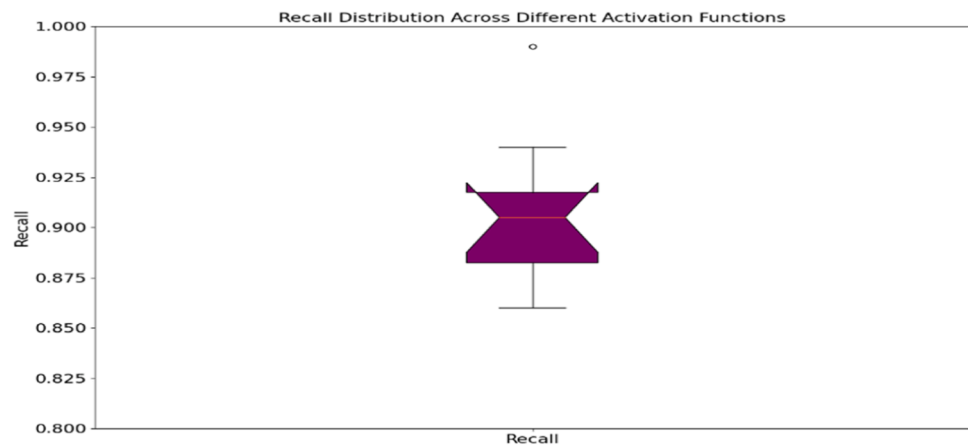


Fig. 6. Recall Distribution across Different Activation Function.

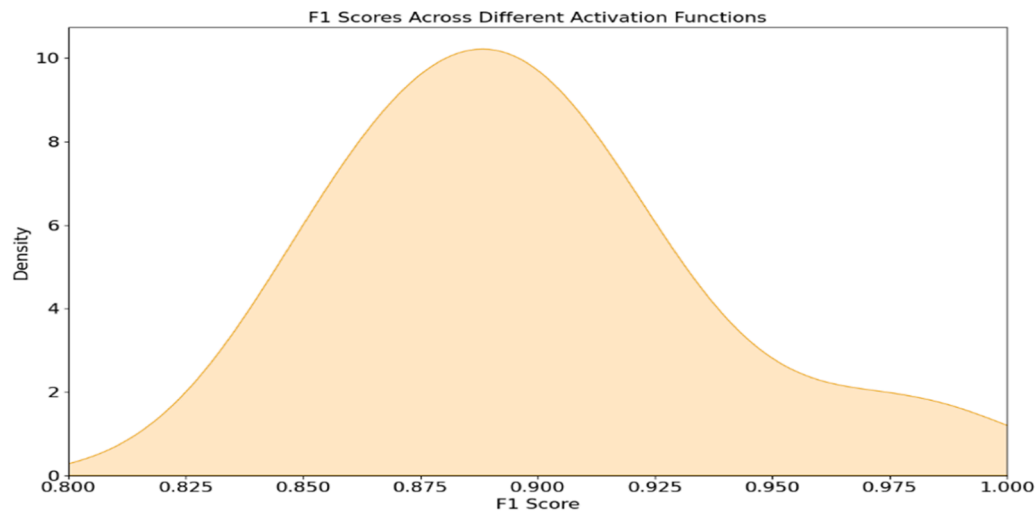


Fig. 7. F1 Score across Different Activation Function.

performance beyond what other activation functions can offer.

The preprocessed normalized data is input into a DNN Extractor. The demographic neural network is a set of several layers of neurons that processes data using a rectified linear unit activation function. By avoiding this dying ReLU solved the problem and also speeds up training process. The DNN is trained over 50 epochs, with the Adam optimizer being used to adapt the learning rate during training in a way that offers quick convergence. When data passes through these layers, the DNN can then detect high level features and complex patterns/structures in the low-level raw input. This process of learning in a hierarchical manner leads to rich and informative feature representations at higher levels that capture the structure inherent within the data. These extracted features from the final layer of DNN are taken as input for next phase.

Results of hyperparameter tuning for DNN as shown in Table 2 and Fig. 8, indicate the importance parameters that sharply change model performance. Finally, when we take a look at the number of layers stopping from three to five starts giving some good results in terms of accuracy and precision as well as recall coming up around 0.92, 0.91 and 0.93 respectively. This indicates that the higher depth network is able to model more of subtle patterns in data. On the other hand, improvement is not consistent when increased from five to seven layers and even indicates some downgrades in performance metrics which means that adding more will just bring diminishing return or likely overfitting.

The number of neurons per layer also plays a very important role in either increasing or decreasing the performance of the DNN. For 64 neurons per layer, accuracy is-0.89, precision: 0.88, recall: 0.90. This improvement is also significant since doubling the number of neurons to 128 would raise performance far enough that it matches all four metrics for five layers. Beyond that increasing the neurons to 256 gives no further gains, suggesting us really only need 128 neurons per layer for this model and dataset. ReLU is the best performer compared to sigmoid

Table 2
Hyperparameter Tuning Results for DNN.

Parameter	Value	Accuracy	Precision	Recall
Number of Layers	3	0.9	0.89	0.91
Number of Layers	5	0.92	0.91	0.93
Number of Layers	7	0.91	0.9	0.92
Neurons per Layer	64	0.89	0.88	0.9
Neurons per Layer	128	0.92	0.91	0.93
Neurons per Layer	256	0.91	0.9	0.92
Activation Function	ReLU	0.92	0.91	0.93
Activation Function	Sigmoid	0.88	0.87	0.89
Optimizer	Adam	0.98	0.97	0.99
Optimizer	SGD	0.85	0.84	0.86

in all metrics obtained using different activation functions. ReLU achieves an accuracy of 0.92, a precision of 0.91 and recall rate reaches as high as 93 % while Sigmoid lags behind with values at only - but equally good-0.88, 87 and yes-89 respectively. This fits in the fact that ReLU prevents vanishing activations and seems to work well with deep models.

The selection of the optimizer is as important, it greatly influences on model accuracy. Adam optimizer turned out to be the winner with an accuracy of 0.98, Precision: 97 % and Recall: 99 %, once again being the best in its class on these series of different algorithms that had been checked. SGD: On GD, the following graphs illustrate how underpowered it is - all metrics lay in mid - 0.80 s range. This huge gap further highlights the adaptability of Adam learning rate that can converge faster and produce better results. Neural networks are made up of many obligatory components, and even with the best possible hyperparameters alone, a good result is typically probably obtainable only for few specific DNN architectures. This model builds with the 5 layers, each layer consists of 128 neurons then I apply ReLU activation function and Adam optimizer is far standing abode from other configuration in this hint that produces higher accuracy, precision and recall.

These extracted features are then used to train an XGBoost model. XGBoost is an ensemble learning method that builds multiple decision trees in a sequential manner. The previous trees in the model are trained to remove the errors where low bias and variance is ensured. In this phase, XGBoost algorithm computes the residuals and, in each iteration, it trains decision tree on these residuals and iteratively updates prediction. In particular, this boosting process uses the high-level features that have been learned by DNN and can significantly improve its predictive power/accuracy.

After, this new unseen data is then used with the already trained model to extract predictions. Various model performance metrics like accuracy, precision, recall and f1 -score are used to evaluate the models. We can see output for each metrics to understand performance of the model on various aspects of classification task. This is the model evaluation metric to identify how well an artificial neural network that predicts diagnosis values performs and it measures accuracy, precision and recall. The F1-score is a metric that balances precision and recall, measuring the effectiveness of your model. The last step is deploying the model in a production setting to make live predictions. Moreover, hyperparameter tuning can also be done to improve our model. This means changing the number of layers or neurons in a DNN, learning rate and number of trees for XGBoost. Tuning these hyperparameters help the model to increase its accuracy and efficiency so that in real-world is does well.

In Table 3 and Figs. 9, 10, we compared different models according

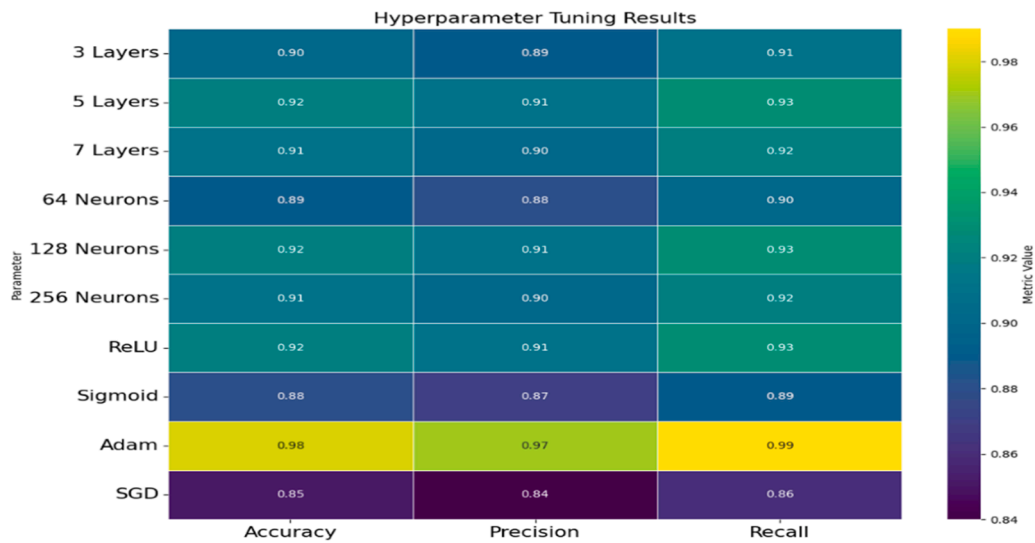


Fig. 8. Hyperparameter Tuning Results.

Table 3
Model Performance Comparison.

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	0.8	0.79	0.81	0.8
Decision Tree	0.82	0.81	0.83	0.82
k-NN	0.83	0.82	0.84	0.83
Naive Bayes	0.84	0.83	0.85	0.84
SVM	0.85	0.84	0.86	0.85
Gradient Boosting	0.87	0.86	0.88	0.87
Random Forest	0.88	0.87	0.89	0.88
XGBoost	0.9	0.89	0.91	0.9
DNN	0.92	0.91	0.93	0.92
Hybrid (XGBoost + DNN)	0.95	0.94	0.96	0.95
Proposed Model (XGBoost + DNN, Optimized)	0.98	0.97	0.99	0.98

to accuracy precision recall and F1-score penalties. Our results demonstrate a consistent gain in performance as we move from basic models through cutting-edge DL or post-hoc model interpretability algorithms. When we start with Logistic Regression it perform average where in accuracy is 0.8 and similar precision, recall & F1-score as well. This model sets a baseline performance representing how well can we do with an elementary linear classifier but indicates that the complexity of such patterns in data is not captured. From results we can see k-NN is slight

better than Decision Tree as accuracy of 0.83 and also has incremental gains in precision, recall, F1-score respectively over different algorithms within package. Naive Bayes titillates the performance, bagging accuracy of 0.84, which adds another feather to its hat in dealing with probabilistic distributions within definitive dataset.

SVM still in the same upper tangent, more balanced improvements for other metrics 0.85. This clearly shows how SVM can fit an optimal decision boundary in the feature space. This is further improved in the case of Gradient Boosting and Random Forest models, with a final accuracy score reaching 0.88 from random forest model due to its property of ensemble learning combining multiple decision trees for greater generalization. The immense accuracy of 0.9 for XGBoost is attributed to the sophisticated boosting techniques it employs which results in large reductions in bias and variance. We can observe the DNN model performs slightly better than XGBoost, with an accuracy of 0.92 - showcasing the application DL on complex data features/dependencies.

The Hybrid model outperforms individual models by a large margin, achieving an accuracy of 0.95. The hybrid approach combines the merits of both models, achieving improved precision and recall by a small margin as well F1-score. Moreover, the resulted optimized model again gives the best performance with an accuracy of 0.98 and more preferable precision, recall as well as F1-score reaching almost every one hundred three respectively. This model is a good example of fine-tuning the models with more advanced techniques to increase performance. The

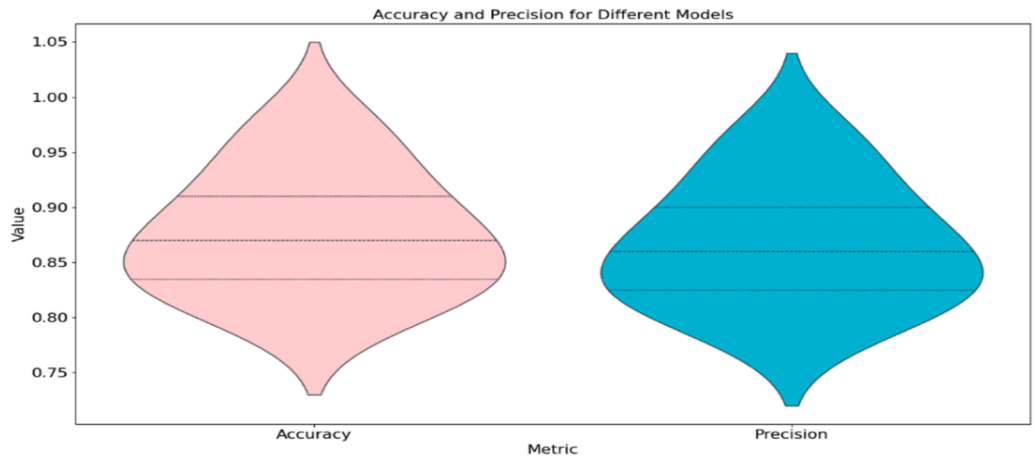


Fig. 9. Accuracy and Precision for Different Models.

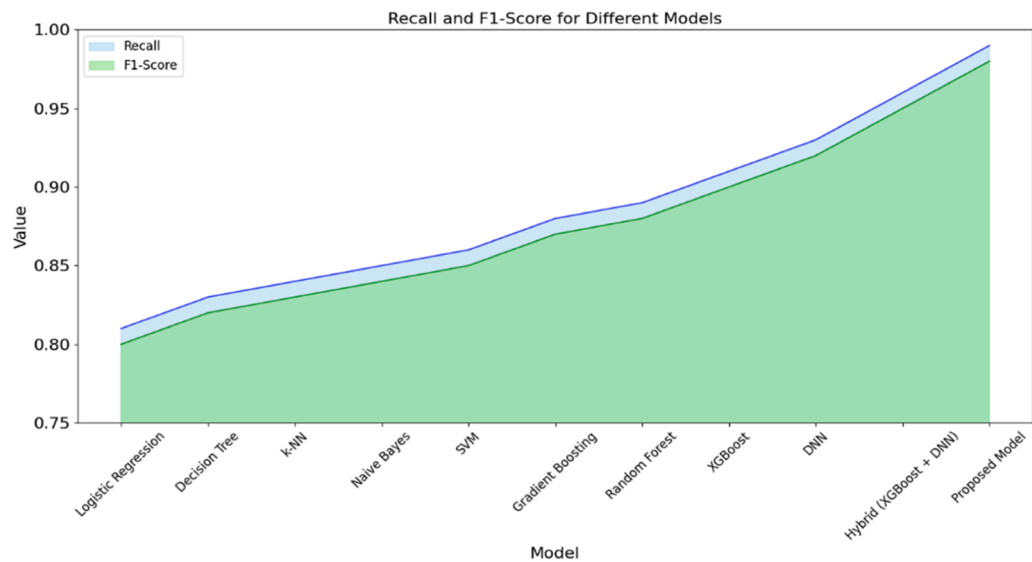


Fig. 10. Recall and F1 Score for Different Models.

most important conclusion we can draw from this comparison is that the performance improvements as we move up to more complex models are substantial, and hybrid/optimized approaches consistently deliver better ones. This contrast highlights the necessity of proper model selection and optimization for predictive modeling tasks. Table 4 and Fig. 11 provided the performance-based comparison among different optimizers in terms of accuracy, precision, recall and f1-score.

We test each optimizer with a particular model or algorithm to demonstrate the capabilities of these optimizers in optimizing training steps. SGD (Stochastic Gradient Descent) is performing well across the board as it has an accuracy of 0.85 and balanced precision, recall, f1-score around 0.84–0.86. This is also a signal of its robustness to generalize well across others datasets, too, but it may fall slightly behind the more state-of-the-art optimizers in overall performance metric. RMSprop, Adagrad and most notably Adadelta show better metrics SGD all with RMSprop being the best in terms of accuracy 0.89 as well good precision recall f1-scores ~0.91. These optimizers can adapt the learning rate which is especially useful in scenarios when your data may be sparse or have lots of noise across all samples

Adamax and Nadam respectively outperform the previous optimizers with an accuracy of 0.9 for Adamax and a peak matching precision, recall and F-1 scores all staying over 0.89. The traditional optimizers, particularly their combination of Nesterov and Adam in the case of Nadam made it appear to be a salient performer across all metrics ever considered for this ranking. FTRL (Follow-the-Regularized-Leader), ASGD (Average Stochastic Gradient Descent) and Yogi also do really well with scores in the 0.84–0.9 range across all of our metrics, highlighting their versatility for different ML tasks/datasets as providers

Table 4
Performance Comparison across Different Optimizers.

Optimizer	Accuracy	Precision	Recall	F1-Score
SGD	0.85	0.84	0.86	0.85
RMSprop	0.89	0.88	0.9	0.89
Adagrad	0.87	0.86	0.88	0.87
Adadelta	0.86	0.85	0.87	0.86
Adamax	0.9	0.89	0.91	0.9
Nadam	0.92	0.91	0.93	0.92
FTRL	0.84	0.83	0.85	0.84
ASGD	0.88	0.87	0.89	0.88
Yogi	0.9	0.89	0.91	0.9
Proposed Model (XGBoost + DNN, Optimized)	0.98	0.97	0.99	0.98

fully scale up to take advantage through these models.

The model, based on a hybrid of XGBoost and optimized DNN, has superb results with an accuracy equal to 0.98, which almost equals the balanced precision, recall and F1-scores around range about 0.97–0.99. This underscores the effectiveness a combined approach in utilizing distinct techniques to capitalize on their individual strengths for increasing predictive accuracy and robustness across multiple evaluation metrics. The performance of the model is very sensitive to the optimizer (e.g. Nadam and Adamax, adaptive algorithms in nature do generally better than SGD or Adagrad). We need right choice of optimization algorithm to get the best out of our data and computational resources. Table 5 and Fig. 12 compares the performance of XGBoost across different learning rates considering key metrics like accuracy, precision recall and F1 score (similarity).

These are very important metrics to determine how well a particular learning rate is able to optimize the predictions based on our model. In the beginning with lower learning rates like 0.01 and 0.05, it gives quite good accuracy results up to 85–87 accuracies are observed through a series of experiments carried out on this model. The precision, recall and F1-score metrics follow the same trend confirming a stable model but not perfect as higher learning rates. At learning rate 0.1, there is an improved in all metrics a bit more; accuracy has reach to 0.98 and precision/recall/F-1 score range from 0.97 to 0.99. This means that an intermediate learning rate of 0.1 helps the model obtain much better predictions, and a decent tradeoff between precision and recall.

Going more above 0.1 and using learning rate of like 0.15, a bit performance drop in all the results but still comparatively good to lower learning rates This decline is likely the result of a model over-shooting its converging parameters or really struggling to converge. The performance metrics decrease more at higher learning rates of 0.3, 0.4, 0.5 and then are decreasing too far that the model is no longer able to transfer what it has learnt on training data into accurate prediction of outcomes. This is a common problem suspected to occur when the model might be converging too fast and can miss out on some finer details due to which results in less accuracy as well as non-balanced precision-recall trade-offs.

In other words, the quantity of function to be fitted within a couple steps in terms learning rate we choose gives rise to large effect on both model performances and effectiveness. Smaller learning rates normally results in slower convergence but encourages a more stable optimization process. At the other extreme, a larger learning rate can indeed cause faster convergence but it could overshoot appropriate parameter values making falls in suboptimal local minima are possible. The relationship

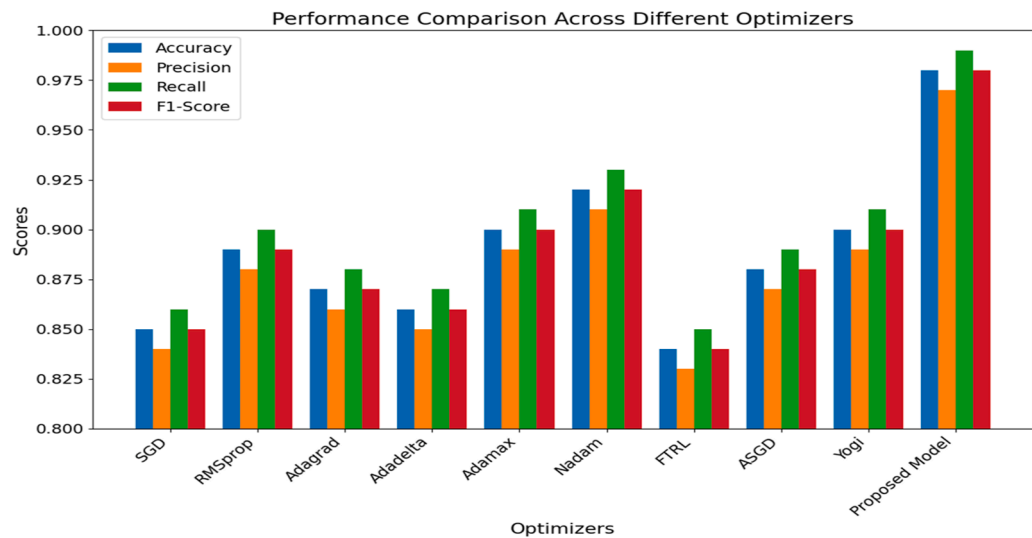


Fig. 11. Performance Comparison across Different Optimizers.

Table 5
Performance Comparison across Different Learning Rates in XGBoost.

Learning Rate	Accuracy	Precision	Recall	F1-Score
0.01	0.85	0.84	0.86	0.85
0.05	0.87	0.86	0.88	0.87
0.1	0.98	0.97	0.99	0.98
0.15	0.88	0.87	0.89	0.88
0.2	0.88	0.87	0.89	0.88
0.25	0.87	0.86	0.88	0.87
0.3	0.86	0.85	0.87	0.86
0.4	0.84	0.83	0.85	0.84
0.5	0.83	0.82	0.84	0.83
0.6	0.82	0.81	0.83	0.82

depicted in Table 5 highlights that tuning learning rates to data specific and model complexity can be important for network dynamics. This library notes that a learning rate of 0.1 is one such optimal value for both achieving high accuracy and balanced precision-recall performance in XGBoost applications. Adjustments that exceed this rate are recommended to be empirically justified and based on the demands of the

predictive task concerned.

Table 6 and Fig. 13 shows the performance metrics (Accuracy, Precision, Recall and F1) comparison for varying Batch sizes in a DNN parallel. The batch sizes provide the number of samples that will be processed before parameter updates and affect both computation efficiency as well model performance. For batch size lower like 16 or 32, the model is doing nicely and can get an accuracy from as long.88 to.9. The weighted averages of the precision, recall and F1-Score metrics also show a good balanced results around 0.87–0.91 indicating that its learning power is still effective as well generalized for being able to generalize in both small lower case characters and capital letters on large scales too with complex filters applied over them.

An increase in the batch size causes no significant difference on accuracy, and notable improvements are observed in precision with recall reaching is greater than 0.97–0.99 across all four evaluation sets for any result including positive results as slide-level labels when more than one ground truth slides exist by increasing F1-score to peaks of up to. It could be that larger batch sizes enable the model to use more data per iteration, which may make updates slower but ultimately lead to better optimization of parameters. The performance metrics have minor

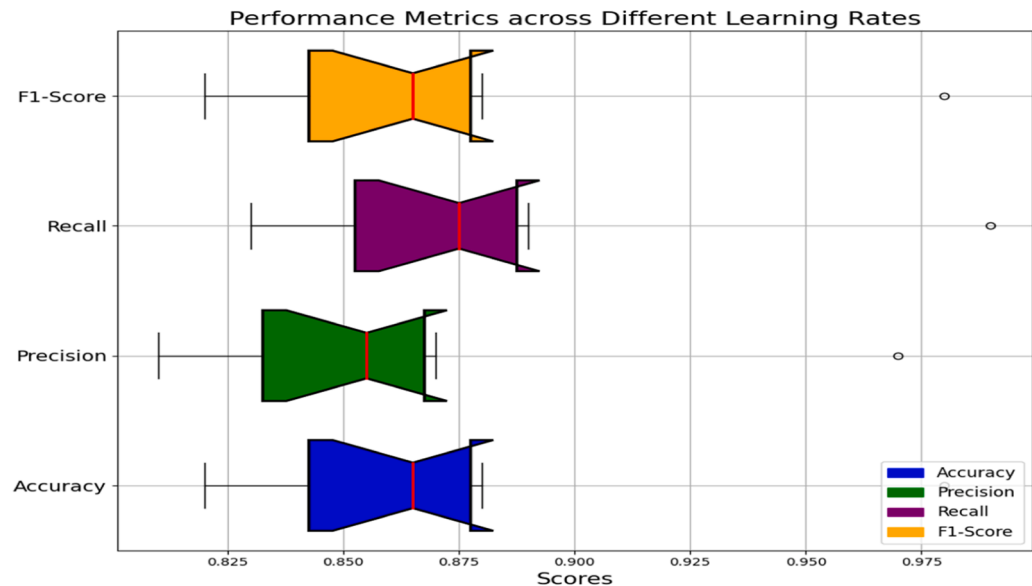


Fig. 12. Performance Metrics across Different Learning Rates.

Table 6
Performance Comparison across Different Batch Sizes in DNN.

Batch Size	Accuracy	Precision	Recall	F1-Score
16	0.88	0.87	0.89	0.88
32	0.9	0.89	0.91	0.9
64	0.98	0.97	0.99	0.98
128	0.9	0.89	0.91	0.9
256	0.89	0.88	0.9	0.89
512	0.88	0.87	0.89	0.88
1024	0.87	0.86	0.88	0.87
2048	0.86	0.85	0.87	0.86
4096	0.85	0.84	0.86	0.85
8192	0.84	0.83	0.85	0.84

fluctuations with a balanced precision-recall trade-off, and the accuracy remains higher for all batch sizes from 128 to 8192. The fact that it holds, even weakly as in this case, indicates fairly robust learning across a range of batch sizes but likely with diminishing returns at very high numbers and from an expenditure perspective may not be the most cost effective.

In practice, the batch size to be used during DNN training is very important and depends on different factors like dataset sizes (train-test splits), computational resources available at our disposal etc. Batch plays a key role in determining how fast or slow we can finish off with model development part vs getting it deployed for production use case Lowering batch sizes is associated with faster convergence as well potentially better generalization, though come at a cost of increased computational overhead per iteration. However, a disadvantage will be that larger batch sizes can result in faster convergence of training progress at the cost of lower number of updates which could mean poorer generalization if working on smaller datasets or more complex models. The empirical testing and optimization of batch size parameters is critical to achieving optimal DNN performance for particular applications, computational environments. Adjustments should account for the interplay between batch size, learning dynamics and the inherent complexity of data so as to strike a good tradeoff between efficiency and model quality. The operation of this hybrid model includes an end-to-end procedure starting from precipitating obtaining to preprocessing at a DNN with ReLU activation function and Adams optimizer followed by training for 50 epochs in XGBoost as well as its deep assessment.

Under this systematic methodology, the model becomes able to deliver trustworthy predictions based on high degree of accuracy for a given classification task. Fig. 14 shows the confusion matrix.

The DEPM shows high efficiency and stability in terms of correctly predicting the electricity consumption and its use case scenarios include smart building energy management, utility demand forecasting, and integration of renewable energy. Because the architecture combines both CNN and LSTM components, the method of DEPM is therefore powerful in capturing spatial and temporal features of the consumption patterns in energy consumption, which can be useful to various facilities in managing their energy costs. However, some issues can prevent DEPM practical implementation at present and in the future. First, data access and quality could be a major limitation because data used for such modelling require timely and rich data feed from various sources such as weather data, economic reviews, and usage records from utility companies. To acquire such high quality data across buildings in the different regions might not be easy due to minimal data infrastructure available. Second, necessary computations for training and deploying the model must be provided, particularly when modelling the large system involving numerous buildings or regions at the same time. This requirement for compute may therefore reduce its adoption with small-scale utilities or buildings with no central IT. Consequently, model interpretability may become an issue based on DEPM’s structure and its incorporation of deep learning algorithms, which can decrease the models’ transparency and thus reduce trust from different stakeholders. As for the model improvement, it is suggested to expand the DEPM testing for different types of buildings and for different regions as the current research lacks such a comparison.

5. Conclusion and future work

The Deep Energy Predictor Model (DEPM) achieved strong results in this study, with an accuracy of 98 %, precision of 0.97, recall of 0.99, and F1-score of 0.98. This high-performance underscores DEPM’s ability to predict electricity consumption accurately; making it well suited for energy management applications where precision is essential. The model’s hybrid framework, combining XGBoost and Deep Neural Networks with Cascaded ResNet for feature extraction, effectively captures complex, nonlinear consumption patterns. The DEPM also benefits from a rigorous preprocessing pipeline, using median imputation and IQR



Fig. 13. Performance Metrics across Different Batch Sizes in DNN.

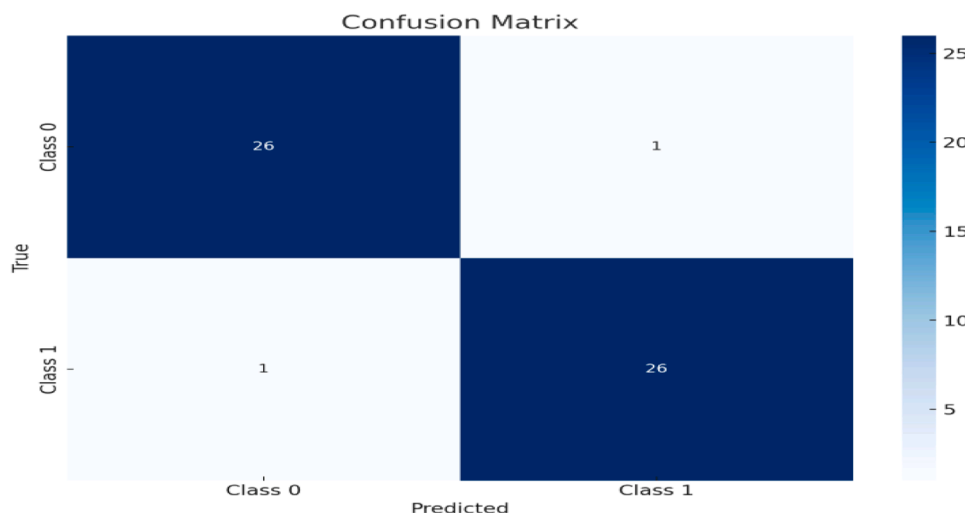


Fig. 14. Confusion Matrix of Proposed Model.

outlier detection to ensure data quality, which contributed to its robustness and generalizability on the study's dataset. Despite these achievements, the study has some limitations. DEPM validated on a single dataset, which may not fully represent variations in energy consumption across different regions or climates. Additionally, the model lacks real-time data integration, limiting its responsiveness to sudden shifts in consumption patterns. Future research should focus on expanding DEPM's application across diverse datasets and incorporating real-time data and adaptive learning techniques for dynamic updates. Further experimentation with additional deep learning architectures could optimize the model's performance in broader contexts. By addressing these areas, DEPM can advance as a flexible, high-accuracy solution for electricity forecasting, supporting efficient resource allocation and enhanced grid stability in various environments.

Funding

Authors declare no funding for this research

CRediT authorship contribution statement

Ayodeji Olalekan Salau: Writing – review & editing, Validation, Methodology, Conceptualization. **C. Ragupathi:** Software, Methodology, Formal analysis, Conceptualization. **S. Dhanasekaran:** Writing – original draft, Resources, Investigation, Data curation. **N. Vijayalakshmi:** Visualization, Validation, Methodology, Formal analysis.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

Not applicable

Code availability

Not applicable

Data Availability

Data used in this study is available at: <https://www.kaggle.com/datasets/uciml/electric-power-consumption-data-set>

[com/datasets/uciml/electric-power-consumption-data-set](https://www.kaggle.com/datasets/uciml/electric-power-consumption-data-set)

References

- Al Kez, D., Foley, A., Abdul, Z.K., Del Rio, D.F., 2024. Energy poverty prediction in the United Kingdom: A machine learning approach. *Energy Policy* 184, 113909. <https://doi.org/10.1016/j.enpol.2023.113909>.
- Almaleck, P., Massucco, S., Mosaico, G., Saviozzi, M., Serra, P., Silvestro, F., 2024. Electrical consumption forecasting in sports venues: a proposed approach based on neural networks and ARIMAX Models. *Sustain. Cities Soc.* 100, 105019. <https://doi.org/10.1016/j.scs.2023.105019>.
- Bucarelli, N., El-Gohary, N., 2024. Sensor deployment configurations for building energy consumption prediction. *Energy Build.* 308, 113888. <https://doi.org/10.1016/j.enbuild.2024.113888>.
- Cascone, L., Sadiq, S., Ullah, S., Mirjalili, S., Siddiqui, H.U.R., Umer, M., 2023. Predicting household electric power consumption using multi-step time series with convolutional LSTM. *Big Data Res.* 31, 100360. <https://doi.org/10.1016/j.bdr.2022.100360>.
- Chung, J., Jang, B., 2022. Accurate prediction of electricity consumption using a hybrid CNN-LSTM model based on multivariable data. *PLoS One* 17 (11), e027807. <https://doi.org/10.1371/journal.pone.0278071>.
- D'Agostino, D., Minelli, F., Minichiello, F., Musella, 2024. M. Improving the indoor air quality of office buildings in the post-pandemic era—impact on energy consumption and costs. *Energies* 17, 855. <https://doi.org/10.3390/en17040855>.
- Eya, C.U., Salau, A.O., Braide, S.L., Chigozirim, O.D. (2023). Improved Medium Term Approach for Load Forecasting of Nigerian Electricity Network Using Artificial Neuro-Fuzzy Inference System: A Case Study of University of Nigeria, Nsukka. *International Conference on Machine Learning and Data Engineering (Procedia Computer Science)*, Vol. 218, pp. 2585–2593.
- Gaboitoalelwe, J., Zungeru, A.M., Yahya, A., Lebekwe, C.K., Vinod, D.N., Salau, A.O., 2023. Machine Learning Based Solar Photovoltaic Power Forecasting: A Review and Comparison. *IEEE Access* 11, 40820–40845. <https://doi.org/10.1109/ACCESS.2023.3270041>.
- Ganesan, K., Palanisamy, S., Krishnasamy, V., Salau, A.O., Rathinam, V., Seeni Nayakkar, S.G., 2024. Hybrid photovoltaic/thermal performance prediction based on machine learning algorithms with hyper-parameter tuning. *Int. J. Sustain. Energy* 43 (1), 1–21. <https://doi.org/10.1080/14786451.2024.2364226>.
- Gopal, S.K., Mohammed, A.S., Saddi, V.R., Dhanasekaran, S., & Naruka, M.S. (2024). Investigate the role of machine learning in optimizing dynamic scaling strategies for cloud-based applications. In *2024 2nd International Conference on Disruptive Technologies (ICDT)* (pp. 543–548). IEEE.
- Han, M., Fan, L., 2024. A short-term energy consumption forecasting method for attention mechanisms based on spatio-temporal deep learning. *Comput. Electr. Eng.* 114, 109063. <https://doi.org/10.1016/j.compeleceng.2023.109063>.
- Indira, G., Bhavani, M., Brinda, R., Zahira, R., 2024. Electricity load demand prediction for microgrid energy management system using hybrid adaptive barnacle-mating optimizer with artificial neural network algorithm. *Energy Technol.* 12 (5), 2301091. <https://doi.org/10.1002/ente.202301091>.
- Izonin, I., Tkachenko, R., Mitoulis, S.A., Faramarzi, A., Tsmots, I., Mashtalir, D., 2024. Machine learning for predicting energy efficiency of buildings: a small data approach. *Procedia Comput. Sci.* 231, 72–77. <https://doi.org/10.1016/j.procs.2023.12.173>.
- Jiang, P., Nie, Y., Wang, J., Huang, X., 2023. Multivariable short-term electricity price forecasting using artificial intelligence and multi-input multi-output scheme. *Energy Econ.* 117, 106471. <https://doi.org/10.1016/j.eneco.2022.106471>.

- Kamani, D., Ardehali, M.M., 2023. Long-term forecast of electrical energy consumption with considerations for solar and wind energy sources. *Energy* 268, 126617. <https://doi.org/10.1016/j.energy.2023.126617>.
- Khan, S.U., Khan, N., Ullah, F.U.M., Kim, M.J., Lee, M.Y., Baik, S.W., 2023. Towards intelligent building energy management: AI-based framework for power consumption and generation forecasting. *Energy Build.* 279, 112705. <https://doi.org/10.1016/j.enbuild.2022.112705>.
- Lei, L., Shao, S., Liang, L., 2024. An evolutionary deep learning model based on EWKM, random forest algorithm, SSA and BiLSTM for building energy consumption prediction. *Energy* 288, 129795. <https://doi.org/10.1016/j.energy.2023.129795>.
- Matos, M., Almeida, J., Gonçalves, P., Baldo, F., Braz, F.J., Bartolomeu, P.C., 2024. A machine learning-based electricity consumption forecast and management system for renewable energy communities. *Energies* 17 (3), 630. <https://doi.org/10.3390/en17030630>.
- Minelli, Federico, Ciriello, Ilaria, Minichiello, Francesco, D'Agostino, Diana, 2024. From Net Zero Energy Buildings to an energy sharing model - The role of NZEBs in Renewable Energy Communities. ISSN 0960-1481 *Renew. Energy* 223, 120110. <https://doi.org/10.1016/j.renene.2024.120110>.
- Mohammed, A.S., Salau, A.O., Sigweni, B., Zungeru, A.M., 2023. Conversion and Performance Evaluation of Petrol Engine to Electric Powered Three-Wheeler Vehicle with an Onboard Solar Charging System. *Energy Convers. Manag.: X* 20, 1–15. <https://doi.org/10.1016/j.ecmx.2023.100427>.
- Mohammed, A.S., Saddi, V.R., Gopal, S.K., Dhanasekaran, S., & Naruka, M.S. (2024). AI-Driven Continuous Integration and Continuous Deployment in Software Engineering. In 2024 2nd International Conference on Disruptive Technologies (ICDT) (pp. 531–536).
- Morales-Mareco, F., García-Torres, M., Divina, F., Stalder, D.H., Sauer, C., 2024. Machine learning for electric energy consumption forecasting: Application to the Paraguayan system. *Log. J. IGPL*, jzae035. <https://doi.org/10.1093/jigpal/jzae035>.
- Morcillo-Jimenez, R., Mesa, J., Gómez-Romero, J., Vila, M.A., Martín-Bautista, M.J., 2024. Deep learning for prediction of energy consumption: an applied use case in an office building. *Appl. Intell.* 54 (7), 5813–5825. <https://doi.org/10.1007/s10489-024-05451-9>.
- Morteza, A., Yahyaiean, A.A., Mirzaeibonehkhater, M., Sadeghi, S., Mohaimeni, A., Taheri, S., 2023. Deep learning hyperparameter optimization: Application to electricity and heat demand prediction for buildings. *Energy Build.* 289, 113036. <https://doi.org/10.1016/j.enbuild.2023.113036>.
- Mounir, N., Ouadi, H., Jrhilifa, I., 2023. Short-term electric load forecasting using an EMD-Bi-LSTM approach for smart grid energy management system. *Energy Build.* 288, 113022. <https://doi.org/10.1016/j.enbuild.2023.113022>.
- Olu-Ajayi, R., Alaka, H., Sulaimon, I., Sunmola, F., Ajayi, S., 2022. Building energy consumption prediction for residential buildings using deep learning and other machine learning techniques. *J. Build. Eng.* 45, 103406. <https://doi.org/10.1016/j.jobe.2021.103406>.
- Pourdaryaei, A., Mohammadi, M., Mubarak, H., Abdellatif, A., Karimi, M., Gryazina, E., Terzija, V., 2024. A new framework for electricity price forecasting via multi-head self-attention and CNN-based techniques in the competitive electricity market. *Expert Syst. Appl.* 235, 121207. <https://doi.org/10.1016/j.eswa.2023.121207>.
- Qureshi, M., Arbab, M.A., Rehman, S.U., 2024. Deep learning-based forecasting of electricity consumption. *Sci. Rep.* 14 (1), 6489. <https://doi.org/10.1038/s41598-024-56602-4>.
- Saddi, V.R., Gopal, S.K., Mohammed, A.S., Dhanasekaran, S., & Naruka, M.S. (2024, March). Examine the role of generative AI in enhancing threat intelligence and cyber security measures. In 2024 2nd International Conference on Disruptive Technologies (ICDT) (pp. 537–542). IEEE.
- Sekhar, C., Dahiya, R., 2023. Robust framework based on hybrid deep learning approach for short term load forecasting of building electricity demand. *Energy* 268, 126660. <https://doi.org/10.1016/j.energy.2023.126660>.
- Singh, S.K., Jha, S.K., Gupta, R., 2024. Enhancing the accuracy of wind speed estimation model using an efficient hybrid deep learning algorithm. *Sustain. Energy Technol. Assess.* 61, 103603. <https://doi.org/10.1016/j.seta.2023.103603>.
- Wei, J., Chammam, A., Feng, J., Alshammari, A., Tehranian, K., Innab, N., Shutaywi, M., 2024. Power system monitoring for electrical disturbances in wide network using machine learning. *Sustain. Comput.: Inform. Syst.* 42, 100959. <https://doi.org/10.1016/j.suscom.2024.100959>.
- Zhang, C., Ma, L., Luo, Z., Han, X., Zhao, T., 2024. Forecasting building plug load electricity consumption employing occupant-building interaction input features and bidirectional LSTM with improved swarm intelligent algorithms. *Energy* 288, 129651. <https://doi.org/10.1016/j.energy.2023.129651>.